

Docker Notes

Dockerfile to running app: architecture, container lifecycle and essential commands with examples

1. Big Picture: Dockerfile to Container

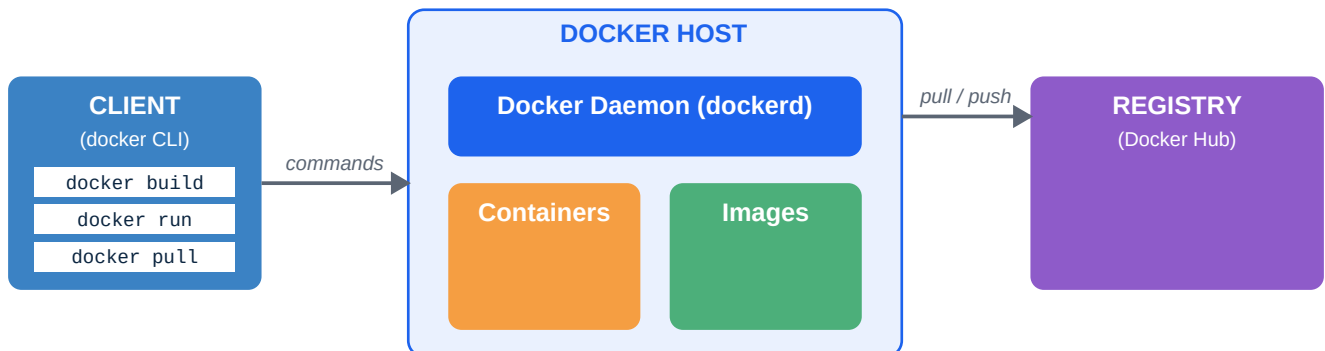
A **Dockerfile** is a text recipe. Building it gives an **image** (a read-only template). Running an image gives a **container** (a live, isolated process). Because the image carries the app plus everything it needs, it runs **the same way in every environment** (laptop, server, cloud).

```
Dockerfile --(docker build)--> Image --(docker run)--> Container
```

Term	Meaning	Analogy
Dockerfile	Instructions to build an image	Recipe
Image	Packaged, read-only template (app + libs + OS layer)	Cake mould / class
Container	Running instance of an image	The cake / object

2. Docker Architecture

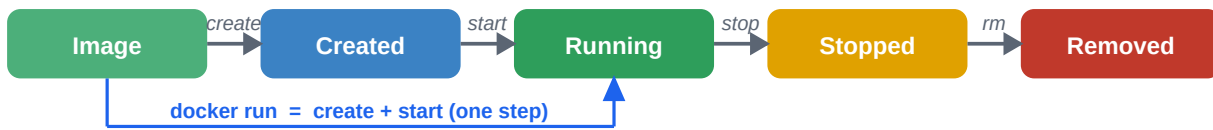
Docker uses a **client-server** model. The client never runs containers itself; it sends requests to the daemon, which does the real work.



Component	What it does
Docker Client	The CLI you type into. Each docker . . . command is sent to the daemon (dockerd) through its API.
Docker Host	The machine (laptop/VM/server) where Docker is installed and containers run.
Docker Daemon (dockerd)	Background service on the host. Builds images, runs and manages containers, talks to registries.
Docker Images	Read-only templates used to create containers. Built from a Dockerfile or pulled from a registry.
Docker Containers	Runnable, isolated instances of images. Can be started, stopped, deleted.
Docker Registries	Storage for images. Docker Hub is the default public one; companies also run private registries.

Note: Example flow: `docker run nginx` -> client asks daemon -> daemon looks for the nginx image locally -> if missing, pulls it from Docker Hub -> creates and starts the container.

3. Container Lifecycle



Know this flow and most commands make sense: **create** makes the container, **start** runs it, **stop** halts it, **rm** deletes it. **run** combines create + start.

4. Core Commands (with examples)

A. Getting and listing images

1. docker pull - download an image

Downloads an image from a registry (Docker Hub by default). If you skip the tag, latest is used.

Syntax:

```
docker pull [options] IMAGE[:tag]
```

Examples:

```
docker pull nginx           # nginx:latest
docker pull ubuntu:20.04    # a specific version (tag)
```

2. docker images - list local images

Shows all images stored on your machine.

Syntax:

```
docker images [options]
```

Examples:

```
docker images               # table: repository, tag, image ID, size
docker images -q            # only image IDs (handy for scripts)
```

B. Container lifecycle commands

3. docker ps - list containers

By default shows only **running** containers.

Syntax:

```
docker ps [options]
```

Examples:

```
docker ps                  # running containers only
docker ps -a               # all containers (running + stopped) (-a = --all)
```

4. docker create - create but do not start

Makes a container from an image but leaves it in the *Created* state.

```
Syntax:
  docker create [options] IMAGE [command] [args]

Examples:
  docker create --name my-container nginx
  # container named my-container, built from the nginx image
```

Careful: The flag is `--name` (two hyphens, no space). Copy-pasting from docs/PDFs often turns it into an en-dash which fails.

5. docker start - start an existing container

Starts one or more containers that already exist but are not running. CONTAINER can be a **name or ID**.

```
Syntax:
  docker start [options] CONTAINER [CONTAINER ...]

Examples:
  docker start my-container          # by name
  docker start container1 container2 # several at once
```

6. docker stop - stop gracefully

Sends a stop signal so the app can shut down cleanly, then force-kills if it takes too long.

```
Syntax:
  docker stop [options] CONTAINER

Examples:
  docker stop my-container
  docker stop -t 30 my-container # wait up to 30 s before force-killing
```

7. docker run - create + start in one step

Most-used command. If the image is not local, Docker pulls it first.

```
Syntax:
  docker run [options] IMAGE [command] [args]

Examples:
  docker run nginx          # runs nginx:latest (blocks your terminal)
  docker run --name cont1 ubuntu:20.04 # give the container a name
  docker run -d nginx        # -d = detached: run in background
  docker run --rm ubuntu echo "hello"  # run once, auto-delete afterwards
```

Option	Meaning
-d	Detached mode: run in the background and print the container ID
--name	Assign your own container name
--rm	Automatically remove the container when it exits

Tip: Use `--rm` for one-off containers (quick tests with curl, ping). Do **not** use it for long-running containers whose data/logs you want to keep.

C. Working inside a container

8. docker exec - run a command in a RUNNING container

Lets you interact with a running container's file system, processes and environment without restarting it.

Syntax:

```
docker exec [options] CONTAINER command [args]
```

Examples:

```
docker exec my-container ls /usr/src/app      # list a folder inside the container
docker exec my-container cat /etc/hostname    # non-interactive command, just prints output
docker exec -it my-container /bin/bash       # open an interactive shell inside
docker exec -it my-container cat /etc/shells  # see which shells are available
```

Flag	Long form	Why it matters
-i	--interactive	Keeps STDIN open so you can send input to the container
-t	--tty	Allocates a pseudo-terminal so output looks like a normal terminal
-it	(both)	Combine them to get a real interactive session (bash, sh)

Note: For simple non-interactive commands (ls, cat, echo) you do not need -it. Use -it only when the command expects keyboard input. If bash is missing, try /bin/sh.

9. docker logs - see container output

Prints what the container's process wrote to stdout/stderr. Essential for debugging and monitoring.

Syntax:

```
docker logs [options] CONTAINER
```

Examples:

```
docker logs cont1          # print logs so far
docker logs -f cont1       # -f = follow: live-stream new log lines (Ctrl+C to exit)
```

10. docker inspect - low-level details

Returns detailed JSON about containers, images, volumes or networks. Works on running *and* stopped containers.

Syntax:

```
docker inspect [options] OBJECT [OBJECT ...]
```

Examples:

```
docker inspect cont1          # name, ID, network config (IP), mounts, state ...
docker inspect nginx:latest   # image ID, creation time, layers and more
```

D. Cleanup commands

11. docker rm - delete containers

Deletes containers. By default it removes only **stopped** ones.

Syntax:

```
docker rm [options] CONTAINER [CONTAINER ...]
```

Examples:

```
docker rm cont1              # remove one
docker rm cont2 cont1        # remove several
docker rm -f cont1           # force: stop and remove even if running
```

12. docker rmi - delete images

Removes one or more images from the local host.

Syntax:
`docker rmi [options] IMAGE [IMAGE ...]`

Examples:
`docker rmi nginx`
`docker rmi ubuntu:20.04`

Tip: An image cannot be removed while a container (even a stopped one) still uses it. Remove the container first with `docker rm`.

13. docker image prune - bulk cleanup

Deletes unused images (by default, dangling ones with no tag) to free disk space.

Syntax:
`docker image prune [options]`

Examples:
`docker image prune` # asks for confirmation
`docker image prune -a` # remove ALL images not used by a container

14. docker --help - built-in manual

Lists every command, or all options of one command.

Syntax:
`docker --help` | `docker COMMAND --help`

Examples:
`docker --help`
`docker run --help`

E. Locating an image

Before pulling or running, find the image you need: browse **hub.docker.com** (check for *Official Image* badges and the available tags), or search from the terminal. Always prefer official images and pin a version tag instead of relying on latest.

```
docker search nginx          # search Docker Hub from the CLI
```

5. Quick Cheat Sheet

Goal	Command
Download an image	<code>docker pull IMAGE[:tag]</code>
List images / IDs only	<code>docker images / docker images -q</code>
List running / all containers	<code>docker ps / docker ps -a</code>
Create only	<code>docker create --name NAME IMAGE</code>
Start / stop	<code>docker start NAME / docker stop NAME</code>
Create + start	<code>docker run [-d] [--name NAME] [--rm] IMAGE</code>
Run command inside	<code>docker exec NAME cmd</code>
Interactive shell inside	<code>docker exec -it NAME /bin/bash</code>
View logs (live)	<code>docker logs [-f] NAME</code>
Detailed info (JSON)	<code>docker inspect NAME_or_IMAGE</code>

Goal	Command
Delete container	<code>docker rm NAME / docker rm -f NAME</code>
Delete image	<code>docker rmi IMAGE</code>
Clean unused images	<code>docker image prune</code>

6. Hands-on Practice: Full Walkthrough

Try these in order. They cover pull, run, inspect, exec, logs and cleanup in one go.

```

docker pull nginx                # 1. download the image
docker images                   # 2. confirm it is on your machine
docker run -d --name web nginx  # 3. start it in the background
docker ps                      # 4. see it running
docker logs web                 # 5. check its output
docker exec -it web /bin/bash   # 6. step inside (type 'exit' to leave)
docker inspect web              # 7. view full details (IP, state ...)
docker stop web                 # 8. stop it
docker ps -a                   # 9. still listed, status = Exited
docker rm web                   # 10. delete the container
docker rmi nginx                # 11. delete the image

```

7. Common Mistakes and Tips

- **Container vs image:** `rm` deletes containers, `rmi` deletes images. Mixing them up is the classic beginner error.
- **Docker only shows running containers by default.** If a container seems to have vanished, run `docker ps -a`.
- **exec needs a running container.** If it is stopped, `docker start` it first.
- **Container name must be unique.** Re-using `--name web` fails until the old one is removed.
- **run vs start:** `run` always creates a *new* container; `start` restarts an existing one.
- **Use IDs or short unique prefixes** if names are long. The first 3-4 characters of a container ID usually work.
- **Pin image versions** (`ubuntu:20.04`) for reproducible builds.